Emergency Notification Network: Explained with a City Traffic Analogy

A plain-language walkthrough of our UDP + Mininet + SDN project, for the group presentation.

The big idea

When something dangerous happens, urgent messages have to reach many people fast, even when the network is jammed. We built a system that sends the alerts and a smart network that clears the road for them.

Who is who

In the project	In the city
Emergency server	Emergency dispatch office
Subscribers (4 clients)	Hospitals and fire stations waiting for orders
An alert message	An ambulance carrying urgent news
Background traffic	Ordinary cars (video streaming, downloads)
The 10 Mbps link between the two switches	A narrow bridge everyone has to cross
Switches	Road junctions
SDN controller	Traffic control room that decides the rules for every junction
UDP	Sending the ambulance straight out with no booking or handshake, so it's fast
ACK (confirmation)	The hospital calling back to say "we got the message"

What happens, step by step

Step 1: The city is built. Mininet creates a fake city inside your laptop. The dispatch office is on one side of the bridge, the hospitals are on the other, and ordinary cars share the same bridge. Nothing is physical, but the roads behave like real ones.

Step 2: Hospitals sign up. Each hospital tells the dispatch office "add me to your list". The office remembers all four. This is the "register multiple subscribers" requirement.

Step 3: The office tells the control room. When a hospital signs up, the dispatch office also sends a quick note to the traffic control room: "a new hospital joined, prepare a fast route to it." The control room then builds a reserved route in advance. This is the link between the application and the network: the app's events change how the network behaves.

Step 4: An emergency happens. The operator triggers an alert. The dispatch office sends one ambulance to each of the four hospitals. The message is marked "emergency" in two ways: it uses a known address and a special urgent sticker on its packets.

Step 5: The control room recognizes the ambulances. Every junction asks the control room what to do with a new vehicle. The control room spots the urgent sticker and the known route and says: "this is an ambulance, send it to the emergency lane." This is classification.

Step 6: The emergency lane. The bridge has two lanes. One is for ordinary cars. The other is a reserved lane with top priority. Ambulances use the reserved lane and skip the jam. This is priority forwarding.

Step 7: Clearing the way. When the alert starts, the control room also squeezes the ordinary lane (from 10 down to 3 Mbps) so the bridge never chokes the ambulances. When the alerts stop for a few seconds, it lets the ordinary lane go back to normal.

Step 8: Confirmation. Each hospital calls back: "message received." If the dispatch office hears nothing within a short time, it sends the ambulance again, up to 3 times. UDP alone doesn't guarantee delivery, so we added this ourselves, but only for serious alerts.

The experiments (what we measured)

Experiment A: rush hour, with and without the smart control room

We flooded the bridge with ordinary cars (more than it can hold) and sent the ambulances through.

- **Without SDN (baseline):** ambulances sit in the same queue as everyone else. Delay went from 0.3 ms to about 70 ms.
- **With SDN:** ambulances used the reserved lane. Delay stayed at about 0.26 ms, as if there were no traffic.

The point is that the same jam hurts the ambulances in one case and doesn't affect them in the other.

Experiment B: bad roads (lost messages)

We made the roads to the hospitals unreliable, so about 1 in 10 ambulances got lost. About 9% of alerts were lost on the first try, which matches what we injected. Because hospitals confirm and the office resends, 0% were lost in the end. The cost was slightly longer delay for the ambulances that had to be resent.

What the numbers mean

- **Latency** is how long the ambulance takes to arrive.
- **Packet loss** is the share of ambulances that never arrive.
- **First-try loss** means lost on the first attempt, before any resend.
- **Residual loss** is what's still missing after the resends. This is the one that matters for safety.

A few terms in plain words

- **SDN (Software Defined Networking):** a central brain (the control room) controls all the junctions, instead of each junction deciding alone.
- **Flow rule:** an instruction the control room gives a junction, like "ambulances go left, in the fast lane."
- **Queue:** a lane on the bridge.
- **Mininet:** software that builds a pretend network on your laptop.
- **Congestion:** too many cars for the bridge.

Suggested split for 4 people

1. **Problem, goal and architecture:** the city map and who is who.
2. **The UDP application:** registration, alerts, confirmations and resends.

3. **The SDN part:** classification, the emergency lane, and the app-to-control-room events.
4. **Experiments and results:** baseline vs SDN, the loss test, and what the numbers show.

Three things to be ready for in the viva

- **Why UDP?** It's fast, and we added reliability only where it's needed.
- **How does the network know what's urgent?** The ports and the urgent sticker, which the server sets.
- **What did you compare against?** The same network with a plain switch and a single lane.